Compte rendu TP 1

1. Étape 1 – Dérivation des clés

Mesure du temps d'Argon2id :

t	m (Mio)	Temps (s)
3	64	0,087
3	128	0,171
4	128	0,244
2	256	0,278
3	256	0,391
4	256	**0,510**
5	256	0,619

Choix retenu : t = 4, m = 256 Mio, p = 4, soit 0,51 s, la valeur la plus proche de l'objectif de 0,5 s. À temps comparable, j'augmente la mémoire plutôt que le nombre de passes, car c'est la mémoire qui limite le plus un attaquant équipé de GPU ou de circuits dédiés. La lenteur est voulue : chaque essai de mot de passe coûte autant à l'attaquant qu'à l'utilisateur légitime.

2. Étape 3 – Tests automatiques

Les tests utilisent des paramètres Argon2 faibles (t = 1, m = 1 Mio, p = 1) uniquement pour rester rapides. Résultat de pytest -v : **9 passed in 0,08 s** .

Test	Comportement attendu	Propriété vérifiée	Résultat
test_aller_retour	Données retrouvées à l'identique	Correction	PASSED
test_probabiliste	Deux chiffrements du même fichier diffèrent	Chiffrement probabiliste (sel et nonce neufs)	PASSED
test_mauvais_mdp	InvalidTag	Authentification de la clé	PASSED
test_octet_modifie	InvalidTag si un octet du chiffré change	Intégrité	PASSED
test_entete_modifie	InvalidTag si le nonce change	Authentification de l'en-tête	PASSED
test_nom_echange	InvalidTag si le nom change	Liaison au contexte (données associées)	PASSED
test_format_inconnu	ValueError	Refus d'un format inconnu	PASSED
test_rechiffrement	Nouveau mot de passe accepté, ancien refusé	Changement de mot de passe	PASSED
test_ancien_fichier_…	Anciens et nouveaux paramètres lisibles	Évolutivité du format	PASSED

Le sujet demandait cinq tests ; j'en ai écrit neuf.

3. Étape 4 – Évolution des paramètres

Pourquoi le format le permet. Chaque fichier contient dans son en-tête la version (CFR1) et les paramètres (t, m, p) qui ont servi à le chiffrer. Au déchiffrement, le code relit ces valeurs au lieu d'utiliser les constantes du programme. On peut donc durcir T, M, P : les nouveaux fichiers utilisent les nouvelles valeurs, les anciens restent lisibles. Si la structure même du format change, on passe à CFR2 et le code reconnaît les deux versions.

Fonction rechiffrer(ancien_mdp, nouveau_mdp, nom, blob). Elle déchiffre avec l'ancien mot de passe, puis rechiffre avec le nouveau, avec un sel et un nonce neufs et les paramètres courants. Un ancien fichier est ainsi migré vers les paramètres renforcés en même temps que son mot de passe change (test_rechiffrement).

4. Questions de réflexion

1. Pourquoi le sel est-il stocké en clair ? Sa publication affaiblit-elle la sécurité ?

Il faut le connaître pour redériver la même clé, donc il est stocké avec le fichier. Son rôle n'est pas le secret mais l'unicité : deux fichiers protégés par le même mot de passe obtiennent des clés différentes, et les tables précalculées deviennent inutiles. Le publier n'affaiblit rien : l'attaquant doit toujours deviner le mot de passe, un essai à la fois, au coût d'Argon2id.

2. Pourquoi le nonce de 96 bits ne risque-t-il pas de se répéter ?

Le sel est neuf pour chaque fichier, donc la clé AES l'est aussi : chaque clé ne chiffre qu'un seul message. Un nonce ne pose problème que s'il se répète sous une même clé, ce qui ne peut pas arriver ici, même avec des millions de fichiers.

3. Que se passerait-il si le nom du fichier n'était pas dans les données associées ?

Un attaquant pouvant écrire dans le dossier échangerait deux fichiers chiffrés avec le même mot de passe, par exemple salaires.cfr et brouillon.cfr. Chacun resterait valide en soi, et l'utilisateur ouvrirait le mauvais contenu sans alerte. Avec le nom authentifié, l'échange provoque un InvalidTag (test_nom_echange).

4. Pourquoi l'en-tête lui-même est-il authentifié ?

Sans cela, un attaquant pourrait modifier sans détection la version, le sel, le nonce ou les paramètres Argon2id, par exemple pour imposer des paramètres très faibles (attaque par rétrogradation). Ici toute modification de l'en-tête fait échouer le déchiffrement (test_entete_modifie). Réserve : les paramètres sont lus avant l'authentification, donc un en-tête malveillant pourrait réclamer une mémoire démesurée ; dans un produit réel, on bornerait t, m et p à la lecture.

5. Pourquoi ne jamais afficher la raison précise d'un échec de déchiffrement ?

AES-GCM ne distingue pas un mauvais mot de passe d'un fichier modifié, et un message détaillé fournirait à un attaquant un oracle pour tester des hypothèses. On affiche un seul message générique, par exemple « Mot de passe incorrect ou fichier corrompu ».